Evidencia GA8-220501096-AA2-EV02. APK (desarrollar módulos móviles según requerimientos del proyecto).

APRENDIZ
KEVIN DANIEL ISAZA

INSTRUCTOR
JULIO CESAR ARGAEZ LENIS

Centro de Gestión Agroempresarial del Oriente
SENA, Vélez, Santander.

FICHA 3186688

2026

Requerimientos Funcionales - Proyecto AgroInventario			
Requerimiento funcional	Descripción del requerimiento funcional	Usuario afectado	Propósito
Registrar usuario	El sistema debe permitir registrar nuevos usuarios.	Administrador	Almacenar la información de los usuarios y sus credenciales de forma segura.
Iniciar sesión	El sistema debe permitir iniciar sesión mediante correo electrónico y contraseña.	Administrador	Permitir el acceso seguro al sistema.
Cerrar sesión	El sistema debe permitir cerrar la sesión del usuario.	Administrador	Finalizar el acceso al sistema de forma segura.
Registrar cultivo	El sistema debe permitir registrar nuevos cultivos.	Administrador	Llevar el control de la producción agrícola.
Consultar cultivos	El sistema debe permitir consultar los cultivos registrados.	Administrador	Facilitar el seguimiento de los cultivos.
Editar cultivo	El sistema debe permitir modificar la información de un cultivo registrado.	Administrador	Mantener actualizada la información de los cultivos.
Eliminar cultivo	El sistema debe permitir eliminar cultivos registrados.	Administrador	Eliminar registros innecesarios.
Registrar trabajador	El sistema debe permitir registrar trabajadores.	Administrador	Administrar la información del personal.
Consultar trabajadores	El sistema debe permitir consultar los trabajadores registrados.	Administrador	Visualizar la información del personal.
Editar trabajador	El sistema debe permitir modificar la información de un trabajador.	Administrador	Mantener actualizados los datos del personal.
Eliminar trabajador	El sistema debe permitir eliminar trabajadores registrados.	Administrador	Mantener organizada la información del personal.
Registrar animal	El sistema debe permitir registrar animales.	Administrador	Llevar el control de los animales de la finca.
Consultar animales	El sistema debe permitir consultar	Administrador	Visualizar la información de los animales.

	los animales registrados.		
Editar animal	El sistema debe permitir modificar la información de un animal registrado.	Administrador	Mantener actualizados los datos de los animales.
Eliminar animal	El sistema debe permitir eliminar animales registrados.	Administrador	Mantener organizada la información de los animales.
Registrar insumo	El sistema debe permitir registrar insumos agrícolas.	Administrador	Mantener actualizado el inventario de insumos.
Consultar inventario	El sistema debe permitir consultar el inventario de insumos y materiales.	Administrador	Verificar la disponibilidad de los recursos.
Editar insumo	El sistema debe permitir modificar la información de un insumo registrado.	Administrador	Actualizar la información del inventario.
Eliminar insumo	El sistema debe permitir eliminar insumos registrados.	Administrador	Mantener organizado el inventario.
Registrar cosecha	El sistema debe permitir registrar las cosechas obtenidas.	Administrador	Llevar el control de la producción agrícola.
Consultar cosechas	El sistema debe permitir consultar las cosechas registradas.	Administrador	Realizar seguimiento a la producción.
Editar cosecha	El sistema debe permitir modificar la información de una cosecha registrada.	Administrador	Mantener actualizados los registros de producción.
Eliminar cosecha	El sistema debe permitir eliminar cosechas registradas.	Administrador	Mantener organizada la información del sistema.
Registrar salario	El sistema debe permitir registrar los salarios de los trabajadores.	Administrador	Llevar el control de los pagos realizados.

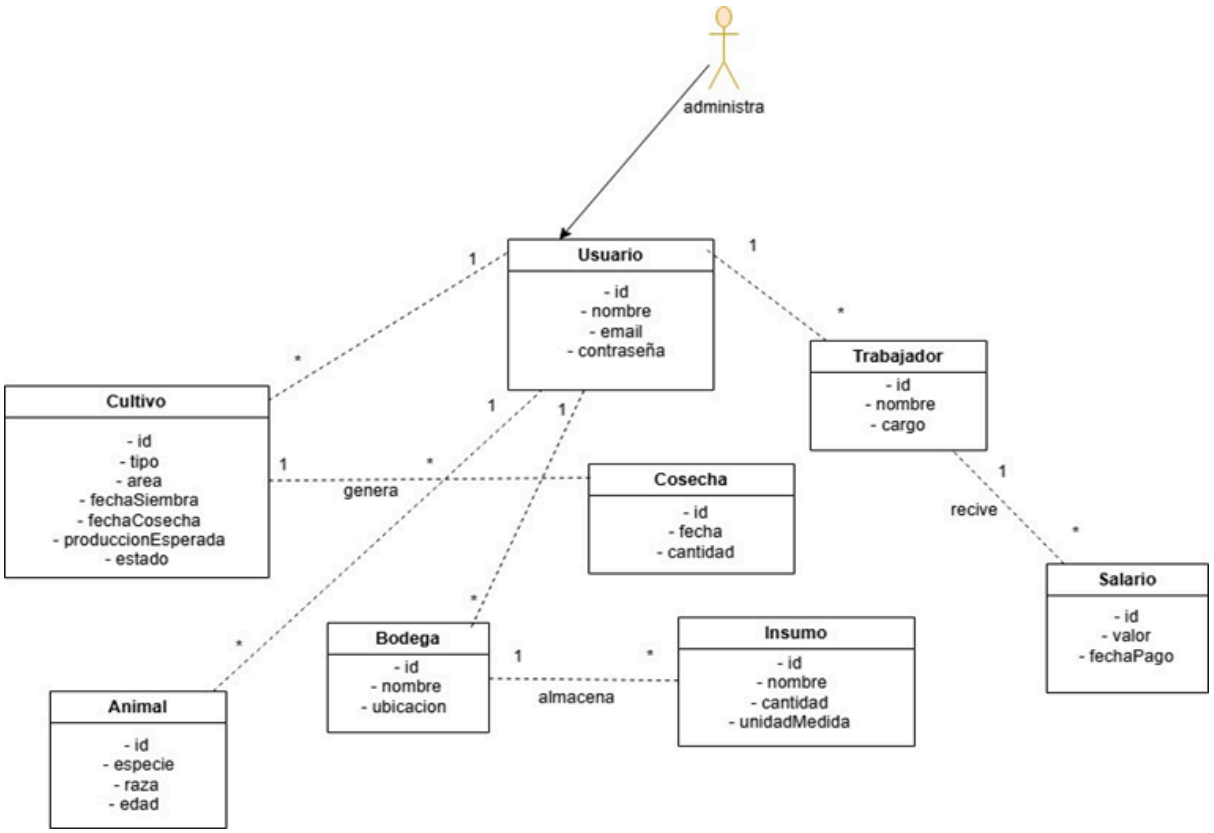
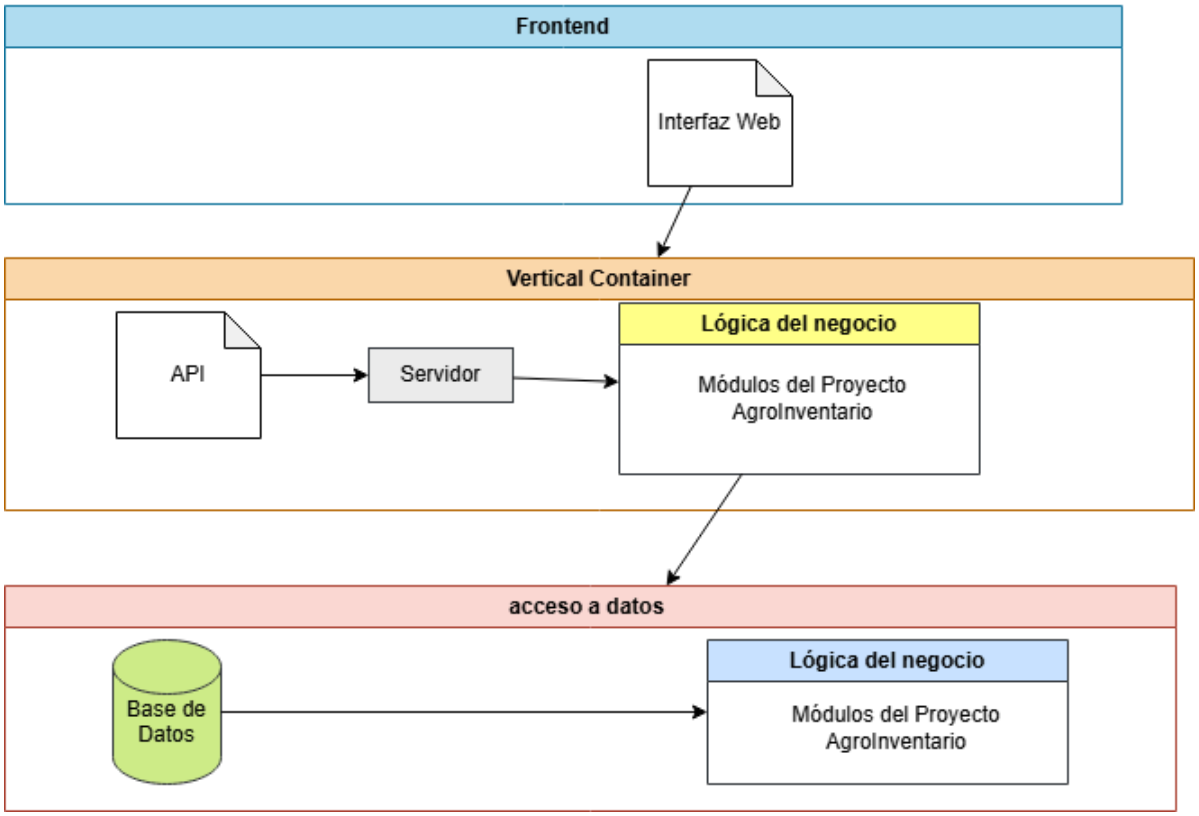
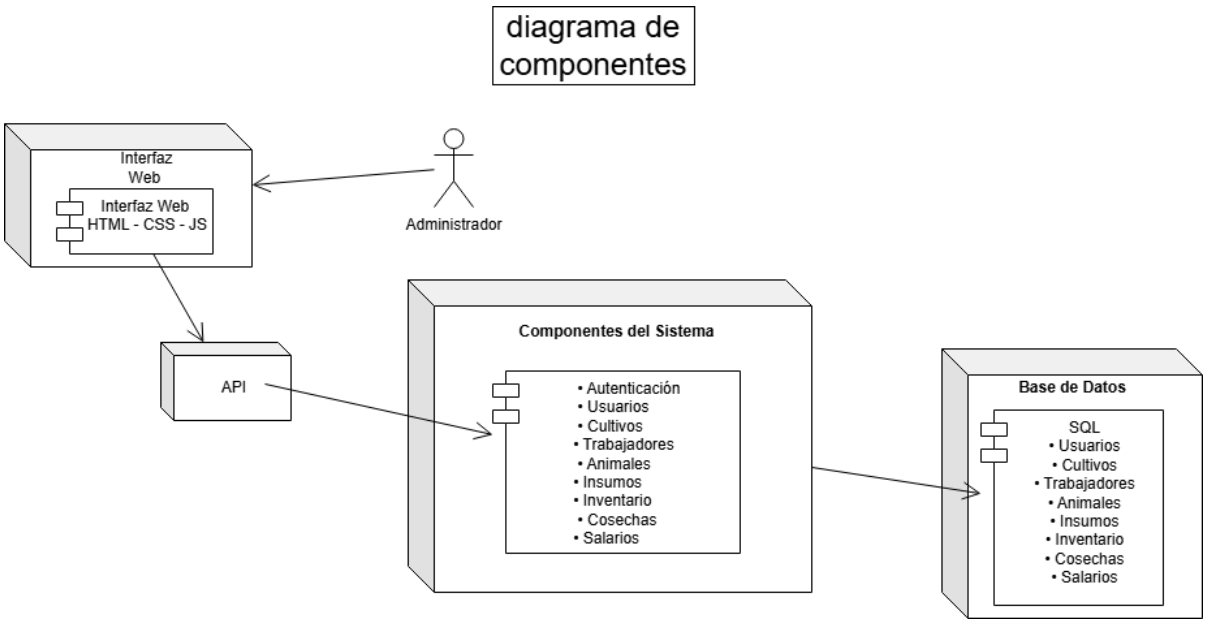


Diagrama de paquetes – Proyecto AgroInventario





capas donde se ubican los componentes de AgroInventario

El proyecto AgroInventario fue desarrollado como una aplicación móvil para dispositivos Android. Para su funcionamiento se utilizaron diferentes tecnologías que se organizan en capas, permitiendo separar la interfaz de usuario, la lógica de la aplicación, los servicios y el almacenamiento de la información.
La aplicación utiliza Android Studio como entorno para generar la aplicación móvil, mientras que los componentes desarrollados anteriormente con HTML, CSS y JavaScript se utilizan dentro de la aplicación. La comunicación con la información almacenada se realiza mediante una API desarrollada con Node.js y Express, conectada a una base de datos MySQL.

Capas de la aplicación

- Android Studio: entorno utilizado para construir y ejecutar la aplicación Android.
- WebView: permite mostrar dentro de la aplicación los archivos web.
- HTML: define la estructura de las pantallas.
- CSS: controla el diseño y apariencia visual.
- JavaScript: permite realizar acciones e interacciones en las pantallas.

Componentes de AgroInventario

- Inicio de sesión.
- Registro de usuarios.
- Recuperación de contraseña.
- Dashboard principal.
- Cultivos.
- Bodega.
- Cosechas.
- Animales.
- Trabajadores.

- Cálculo de salarios.

Capa de lógica de aplicación
Esta capa se encarga de procesar las acciones realizadas por el usuario y establecer la comunicación con el servidor utilizando archivo JavaScript.

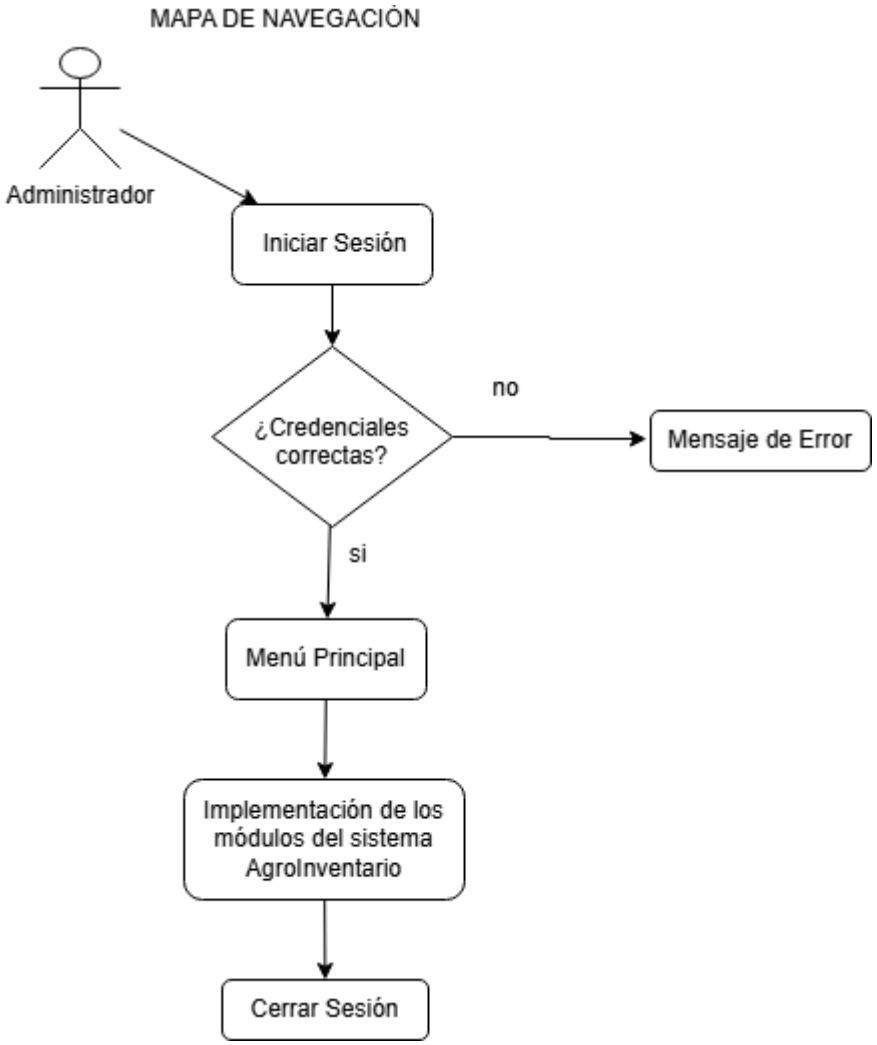
Capa de servicios
La capa de servicios permite la comunicación entre la aplicación Android y la base de datos.
- Node.js
- Express
- API REST
- Endpoints de la aplicación.

Capa de datos
La capa de datos es donde se almacena la información utilizada por AgroInventario.
- MySQL como sistema gestor de base de datos.
- phpMyAdmin para administrar y consultar la información.
- Base de datos AgroInventario.
- Tablas de información, como usuarios y cultivos.

Metodología de desarrollo de software
Para el desarrollo del proyecto AgroInventario se tomó como referencia una metodología de desarrollo de software que permite organizar el proceso de construcción de la aplicación desde la identificación de las necesidades hasta las pruebas y puesta en funcionamiento. Para este proyecto se puede utilizar la metodología Scrum, ya que permite desarrollar el sistema de manera incremental, dividiendo el trabajo en diferentes funcionalidades o módulos.

Etapa	**Actividad realizada en AgroInventario**
Planificación	Se identificaron las necesidades del sistema para administrar información agrícola.
Análisis	Se determinaron los usuarios, cultivos, bodega, cosechas, animales, trabajadores y salarios que manejaría el sistema.
Diseño	Se diseñó la interfaz utilizando HTML y CSS, además de definir la estructura de la base de datos.
Desarrollo	Se programaron las funcionalidades utilizando JavaScript y se realizó la conexión con el servicio web y la base de datos MySQL.
Pruebas	Se verificó el registro e inicio de sesión, almacenamiento de cultivos, consulta de información, eliminación y actualización de registros.

Implementación	La aplicación fue adaptada para ejecutarse desde Android Studio y comunicarse con el servidor mediante la red local.
Mantenimiento	Se pueden realizar correcciones y agregar nuevas funcionalidades según las necesidades del usuario.



Codificar cada módulo en el lenguaje seleccionado

Para el desarrollo de la aplicación AgroInventario se seleccionó JavaScript como lenguaje principal para la lógica de la aplicación, acompañado de HTML para la estructura de las interfaces y CSS para el diseño visual. La aplicación se integró con un servicio web encargado de comunicarse con la base de datos MySQL.

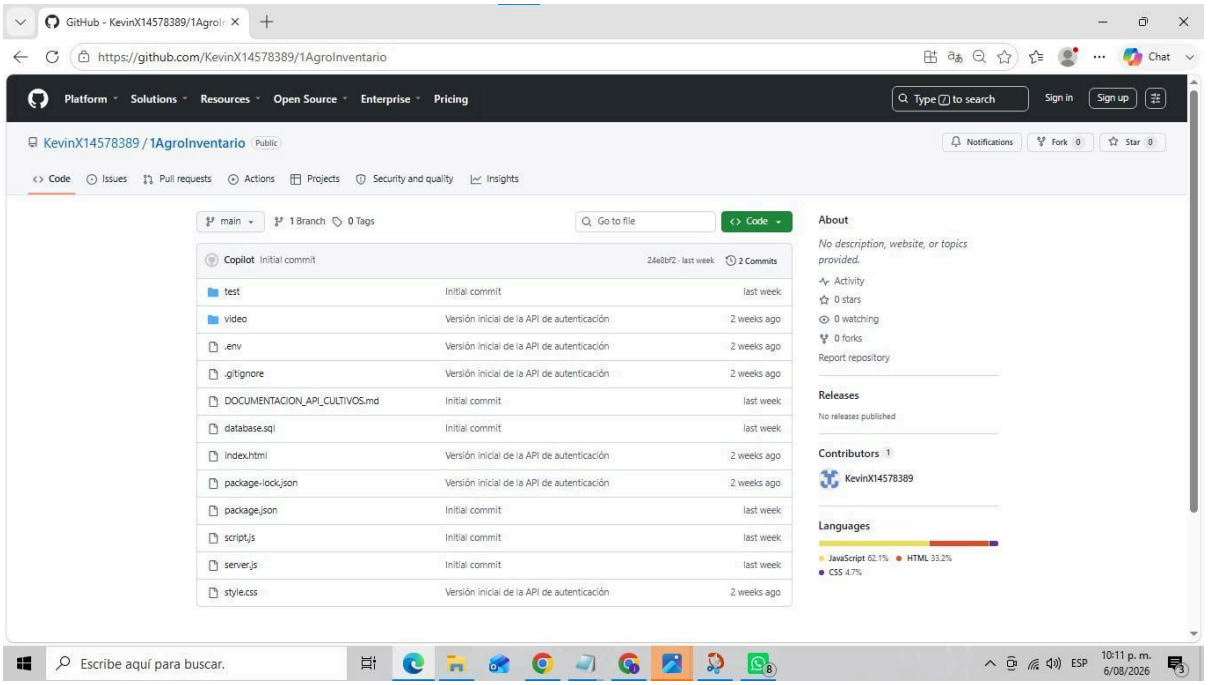
Módulo	Funcionalidades	Tecnología utilizada
Inicio de sesión	Validar correo y contraseña e ingresar al sistema	HTML, CSS, JavaScript, API
Registro de usuarios	Registrar nuevos usuarios en la base de datos	HTML, JavaScript, API, MySQL
Cultivos	Registrar, consultar, editar y eliminar cultivos	HTML, CSS, JavaScript, API, MySQL
Bodega	Registrar y consultar insumos y productos	HTML, CSS, JavaScript
Cosechas	Registrar productos cosechados, kilos y fechas	HTML, JavaScript
Animales	Registrar tipo, cantidad y edad de los animales	HTML, JavaScript
Trabajadores	Registrar empleados, cédula y cargo	HTML, JavaScript
Salarios	Calcular salarios, horas extras y descuentos	HTML, JavaScript
Calculadora agrícola	Calcular densidad de plantas y convertir m² a hectáreas	JavaScript
Reportes	Generar información de inventario	JavaScript y jsPDF

Manejo de un repositorio de control de versiones (Git y GitHub)

Para el desarrollo del proyecto AgroInventario Web se utilizó Git como sistema de control de versiones y GitHub como plataforma para alojar el repositorio del proyecto.
El uso de Git y GitHub permitió llevar un control de los cambios realizados durante el desarrollo de la aplicación, mantener un historial de las modificaciones del código y almacenar una copia de seguridad del proyecto en la nube.



link del repositorio :https://github.com/KevinX14578389/1AgroInventario



Determinar las librerías necesarias para la capa de presentación
En el proyecto AgroInventario se determinaron las librerías y tecnologías necesarias para construir la capa de presentación de la aplicación. Esta capa se encarga de mostrar las interfaces, formularios, botones, tablas, módulos y elementos visuales con los que interactúa el usuario.

Tecnología / librería	Función en AgroInventario
HTML5	Construcción de la estructura de las pantallas, formularios, tablas, botones y módulos.
CSS3	Diseño visual, colores, tamaños, distribución y adaptación de la interfaz a diferentes dispositivos.
JavaScript	Control de eventos, validaciones, navegación entre módulos, cálculos y comunicación con el servicio web.
jsPDF	Generación de documentos y reportes en formato PDF desde la aplicación.
Fetch API	Comunicación entre la capa de presentación y el servicio web mediante solicitudes HTTP.
DOM API	Permite modificar dinámicamente los elementos HTML desde JavaScript.

Determinar los frameworks a utilizar

Para el desarrollo de AgroInventario se seleccionaron tecnologías que permiten construir la aplicación, desarrollar la interfaz y establecer la comunicación con la base de datos.

Framework / tecnología	Uso en AgroInventario
Android Studio	Entorno de desarrollo utilizado para preparar y ejecutar la aplicación en dispositivos Android.
Node.js	Entorno utilizado para ejecutar el servidor y el servicio web.
Express.js	Framework utilizado para crear la API REST que comunica la aplicación con la base de datos.
HTML5	Estructura de las interfaces de la aplicación.
CSS3	Diseño y presentación visual.
JavaScript	Lógica de la aplicación, eventos, cálculos y comunicación con la API.
MySQL	Sistema gestor de base de datos utilizado para almacenar la información.

División de los módulos en componentes reutilizables

En el desarrollo de AgroInventario se recomienda dividir los diferentes módulos en componentes independientes y reutilizables. Esto permite organizar mejor el código, facilitar el mantenimiento y evitar la repetición de funciones.

Componente	Función	Módulos donde puede reutilizarse
Formulario	Permite ingresar información	Cultivos, bodega, cosechas, animales y trabajadores
Tabla de datos	Muestra los registros almacenados	Todos los módulos
Botón Guardar	Envía la información del formulario	Todos los módulos
Botón Editar	Permite modificar registros	Cultivos y otros registros
Botón Eliminar	Elimina un registro	Cultivos, bodega, animales, trabajadores y salarios
Modal	Muestra formularios o información adicional	Insumos, labores y edición de cultivos
Validación de formularios	Verifica que los datos estén completos	Registro, cultivos, trabajadores, etc.
Navegación	Permite cambiar entre módulos	Toda la aplicación

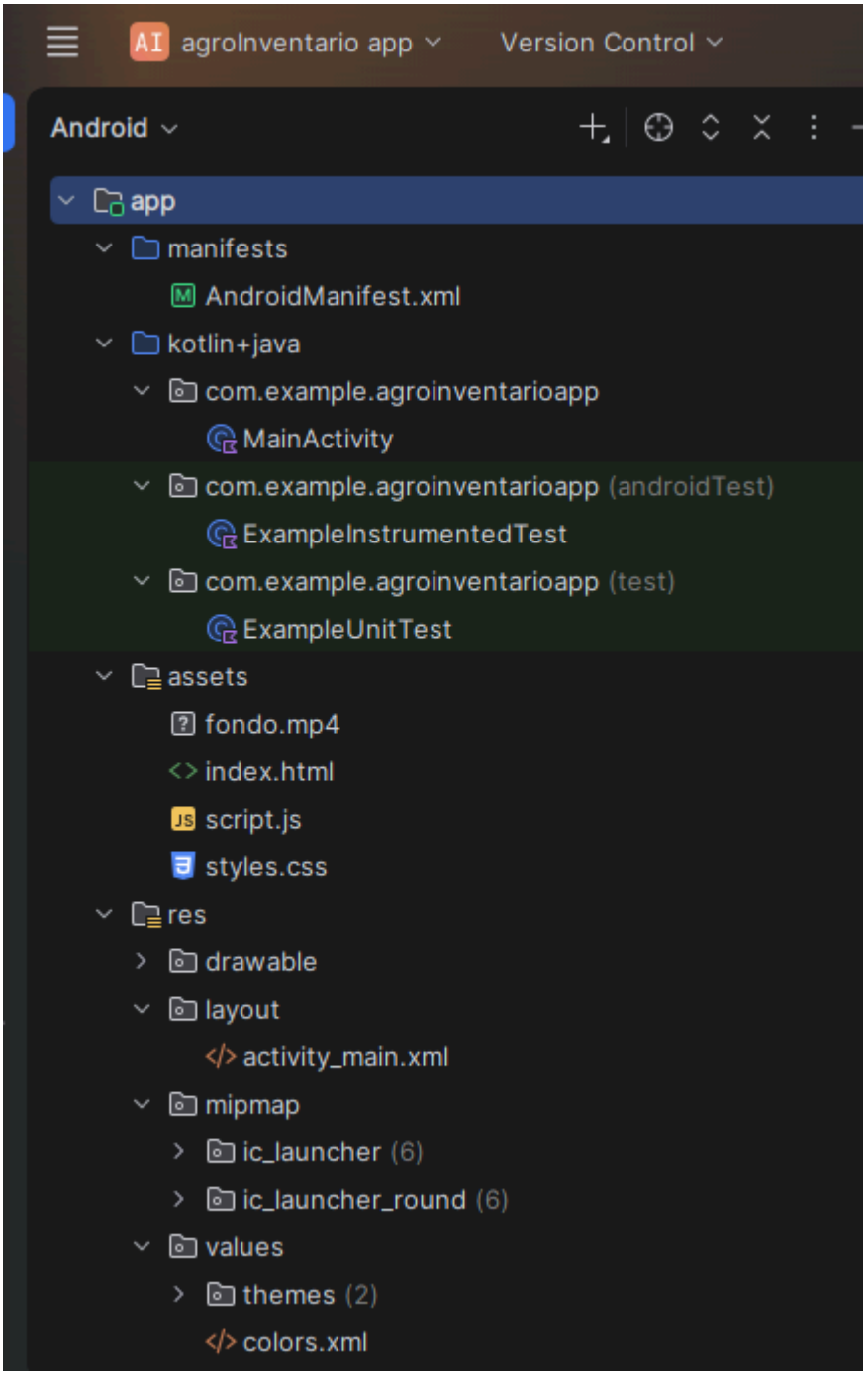
Cálculos	Realiza operaciones automáticamente	Densidad de siembra y salarios

Aplicar la codificación siguiendo las buenas prácticas de escritura de código
Durante el desarrollo de AgroInventario se aplicaron buenas prácticas de programación con el objetivo de obtener un código organizado, comprensible, reutilizable y fácil de mantener.

Buena práctica	**Aplicación en AgroInventario**
Nombres descriptivos	Se utilizaron nombres como registerUser(), loginUser(), saveCrop() y renderCrops().
Organización por funciones	Las funcionalidades se separaron en funciones independientes para cada operación.
Comentarios	Se agregaron comentarios para identificar secciones como autenticación, cultivos, bodega y empleados.
Reutilización de código	Se utilizaron funciones reutilizables como deleteItem() y showModule().
Validación de datos	Se verifican campos obligatorios antes de enviar información al servidor.
Manejo de errores	Se utilizaron bloques try/catch para controlar errores de conexión y solicitudes.
Código estructurado	El proyecto se dividió en archivos como HTML, CSS y JavaScript.
Uso de constantes	Se definieron constantes como API_URL y API_BASE para evitar repetir las direcciones de la API.
Indentación	El código mantiene una estructura visual que facilita su lectura.
Programación asíncrona	Se utilizaron async y await para las operaciones con el servicio web.

División del código fuente en paquetes

Para el proyecto **AgroInventario** se recomienda organizar el código fuente en diferentes carpetas y archivos de acuerdo con la función que cumple cada componente. Los nombres utilizados deben ser claros y fáciles de identificar, permitiendo encontrar rápidamente el código correspondiente a cada módulo.


agroInventario app
Version Control
Android
app
manifests
AndroidManifest.xml
kotlin+java
com.example.agroinventarioapp
MainActivity
com.example.agroinventarioapp (androidTest)
ExampleInstrumentedTest
com.example.agroinventarioapp (test)
ExampleUnitTest
assets
fondo.mp4
index.html
script.js
styles.css
res
drawable
layout
activity_main.xml
mipmap
ic_launcher (6)
ic_launcher_round (6)
values
themes (2)
colors.xml


data_extraction_rules.xml
Gradle Scripts
build.gradle.kts (Project: agroInventario_app)
build.gradle.kts (Module :app)
proguard-rules.pro (ProGuard Rules for ":app")
gradle.properties (Project Properties)
gradle-wrapper.properties (Gradle Version)
libs.versions.toml (Version Catalog "libs")
local.properties (SDK Location)
settings.gradle.kts (Project Settings)

Aplicar los patrones de diseño de acuerdo con la arquitectura del software por componente

En el proyecto AgroInventario se aplicó una arquitectura por componentes, separando la interfaz de usuario, la lógica de negocio, el servicio web y la base de datos. Para facilitar la organización y el mantenimiento del código se tomó como referencia el patrón MVC.

Vista Interfaz de usuario

La Vista corresponde a los elementos que el usuario puede observar e interactuar.
En AgroInventario está representada principalmente por:

- index.html
- style.css
- Formularios
- Botones
- Tablas
- Ventanas modales
- Menús de navegación

Lógica de la aplicación

El Controlador se encarga de recibir las acciones del usuario y ejecutar las operaciones correspondientes.
En AgroInventario esta responsabilidad se encuentra principalmente en JavaScript.

Datos

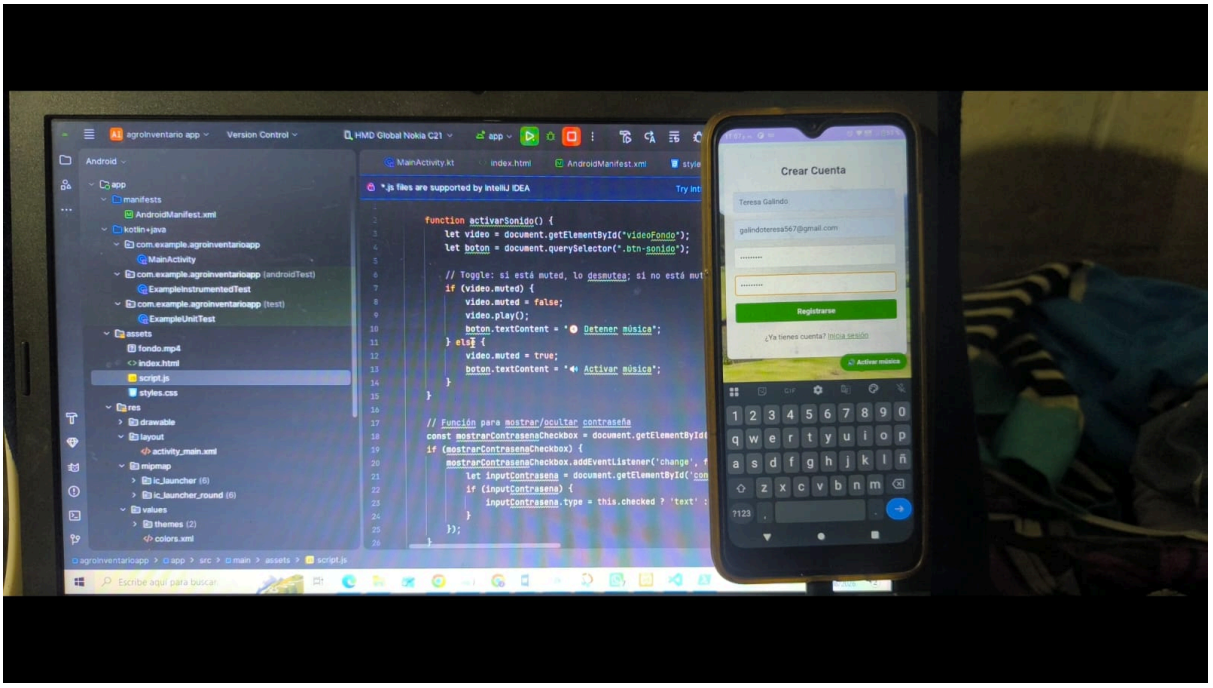
El Modelo corresponde a la información que maneja el sistema y su almacenamiento.
MySQL
para almacenar información como:

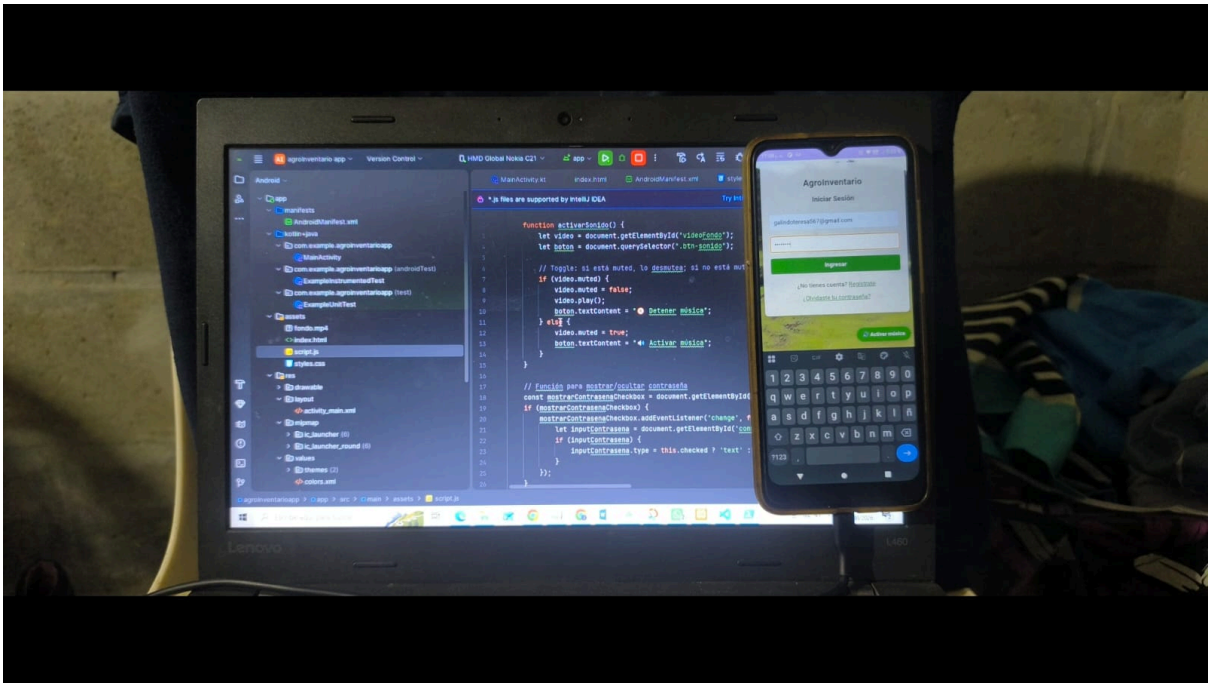
- Usuarios
- Cultivos
- Cosechas
- Trabajadores

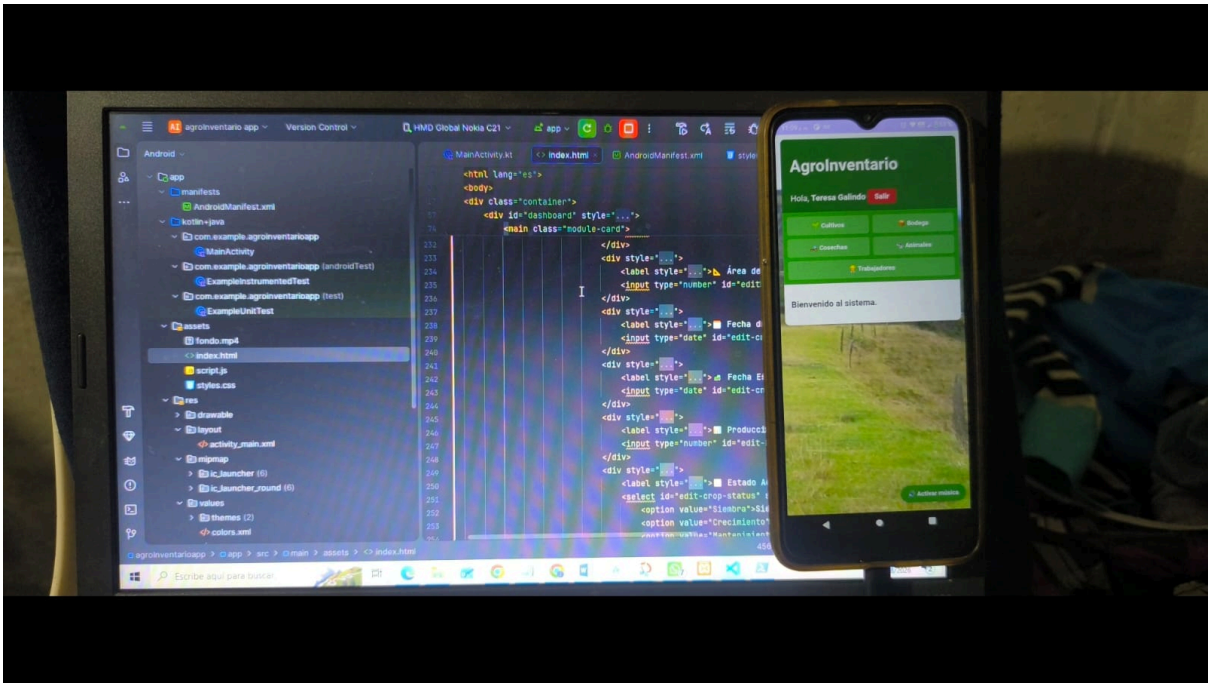
Patrón utilizado

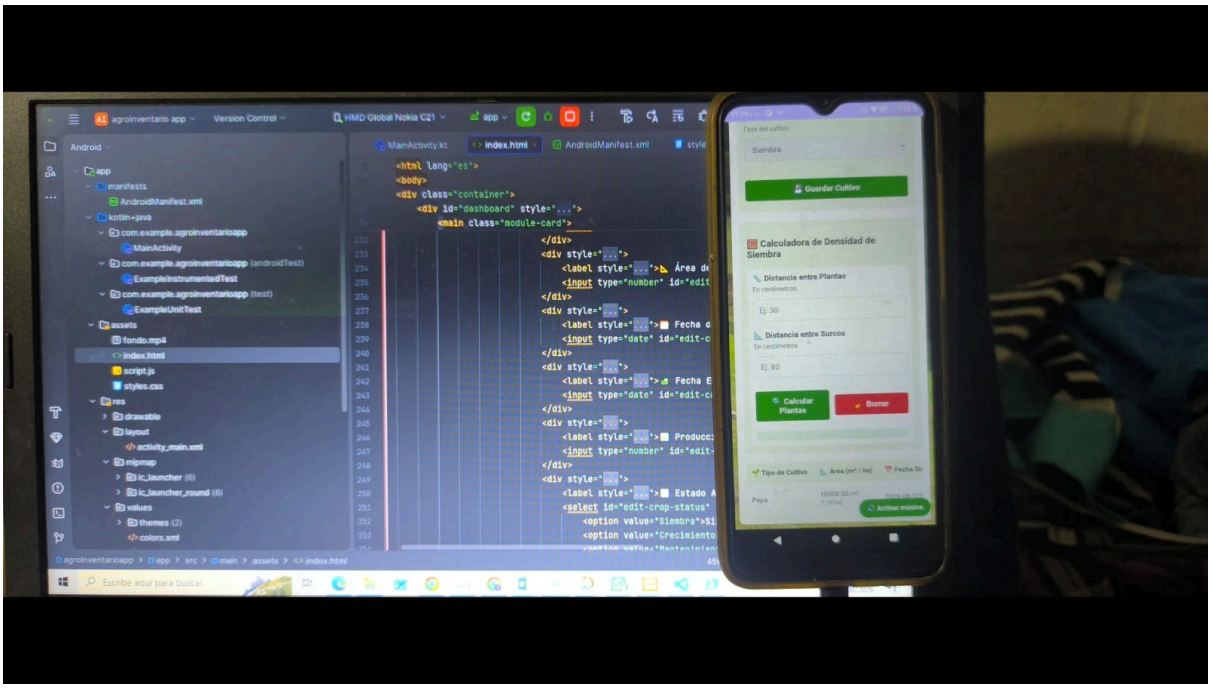
Componente	**AgroInventario**
Modelo	Base de datos MySQL
vista	HTML + CSS
Controlador	JavaScript + API
servidor	Node.js + Express.js

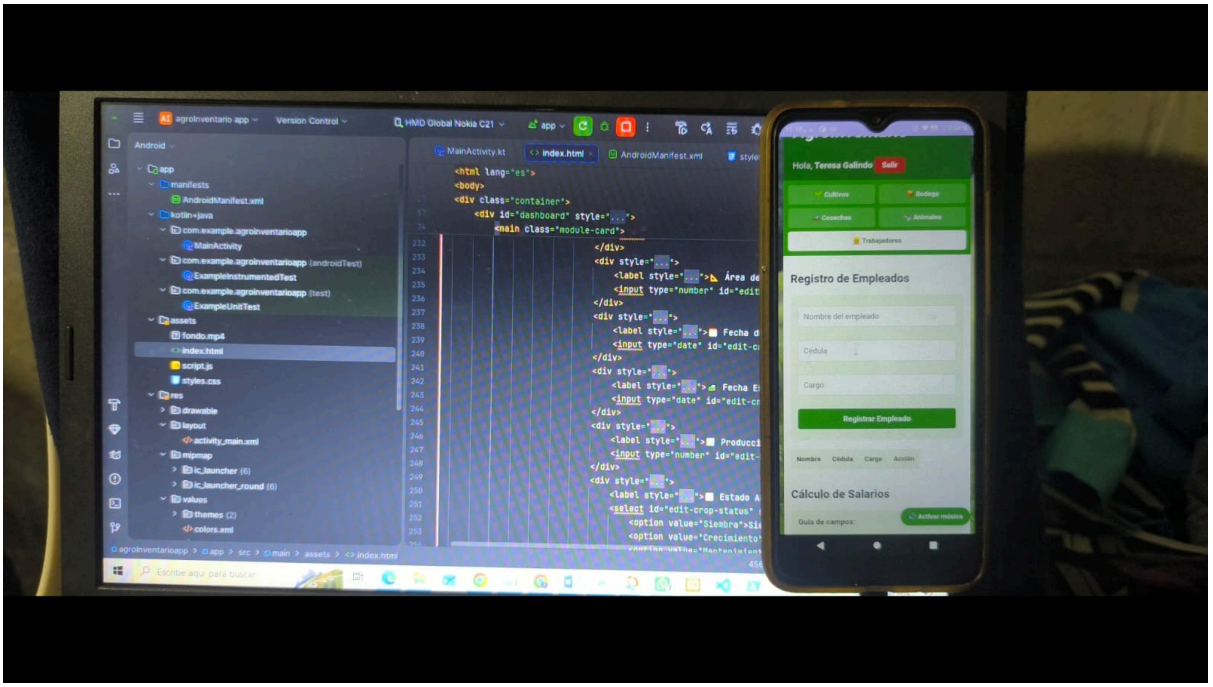
Pruebas unitarias de AgroInventario






AgroInventario
Hola, Teresa Galindo
Salir
Bienvenido al sistema.


AgroInventario
Hola, Teresa Galindo
Salir
Bienvenido al sistema.



deja crear cuenta y acceder toso los modulos del proyecto se visualiza el proyecto en Android Studio, donde se gestiona la estructura de directorios (app/src/main/assets). Esto confirma que el desarrollo está correctamente organizado y listo para la compilación o despliegue en dispositivos Android.
Demostrar la operatividad y navegabilidad total de la aplicación, confirmando que el usuario puede acceder, interactuar y utilizar todos los módulos registrados en el sistema de manera independiente.

Infraestructura de Servidores y Gestión de Datos
En cuanto a la arquitectura técnica del proyecto, hemos implementado un modelo de servicio web con conectividad local que garantiza la gestión eficiente de la información. Esto se compone de tres pilares.

1. El Servidor (Backend): En la segunda imagen (la terminal negra), se ve el servidor corriendo en Node.js con el mensaje "Servidor escuchando en http://localhost:3000" y los endpoints habilitados (/api/auth/register, /api/auth/login, /api/cultivos, etc.). Este servidor es el puente que recibe las peticiones HTTP.
2. La Interfaz y el Acceso a Datos (Frontend): En la primera imagen, dentro de Android Studio, se observa el archivo index.html cargado en los recursos (assets). Cuando el usuario interactúa con módulos como el de Bodega (que se ve abierto en el código), la aplicación utiliza JavaScript (script.js) para conectarse mediante la IP de la red local al servidor.
3. Gestión de la Información: Mediante esta arquitectura, cada vez que el usuario guarda, edita o consulta datos (como cultivos, bodega o empleados), el sistema hace la petición a la API, la cual procesa la información y la almacena o recupera directamente de la base de datos en MySQL."